

SUPPORTSPHERE

Customer Support Ticket Management System

Capstone Project Report

Developed By: Chinnii Sushma

1. Executive Summary

SupportSphere is a web-based Customer Support Ticket Management System developed using ASP.NET Core MVC, Entity Framework Core, and SQL Server. The system provides a centralized platform for customers to raise support tickets, track issue status, submit feedback, and communicate with administrators. Administrators can efficiently manage, resolve, and monitor customer issues through a dedicated dashboard.

The project automates the support process and improves customer satisfaction by providing transparency and efficient issue tracking.

2. Problem Statement

Many organizations still handle customer issues through emails, spreadsheets, or manual processes. These methods often result in delayed issue resolution, poor tracking, lack of transparency, and difficulty generating reports.

SupportSphere addresses these challenges by providing a structured ticket management platform that enables efficient communication between customers and support teams.

3. Project Objectives

- Provide an online platform for raising support tickets.
- Enable customers to track ticket status.
- Allow administrators to manage and resolve tickets.
- Collect customer feedback after ticket resolution.
- Maintain a centralized repository of support requests.
- Improve customer support efficiency and transparency.

4. Technology Stack

Frontend

- Razor Views
- HTML5
- CSS3
- Bootstrap 5
- JavaScript

Backend

- ASP.NET Core MVC
- C#

Database

- SQL Server
- Entity Framework Core

Authentication & Security

- ASP.NET Core Identity
- Role-Based Authorization

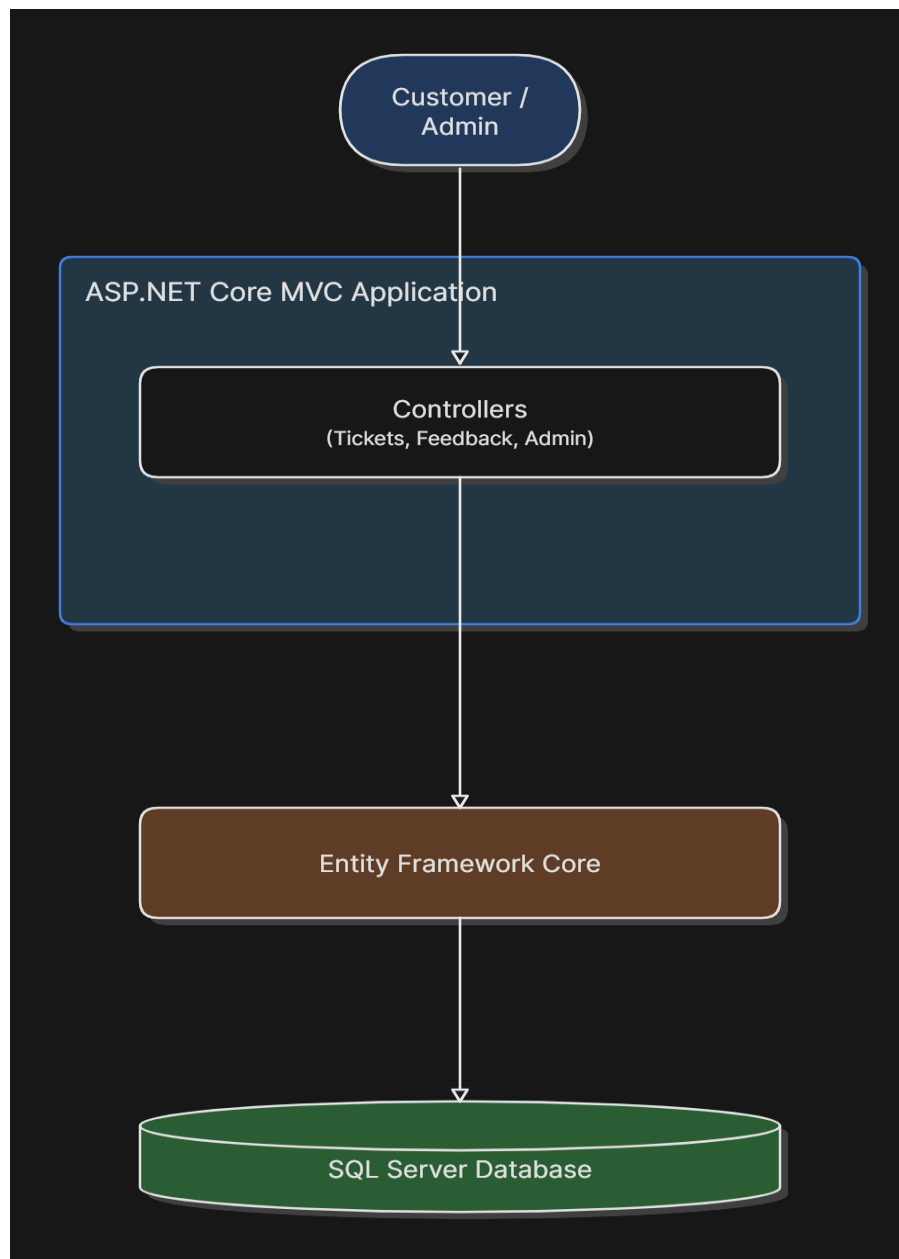
API Documentation

- Swagger / OpenAPI

Testing

- xUnit

5. System Architecture Diagram



Architecture Workflow

1. Customer or Admin logs into the SupportSphere application and accesses the required features based on their role.
2. The request is sent to the Controller (TicketsController, AdminController, or FeedbackController), which processes the user's action.
3. Entity Framework Core handles database operations and converts application data into SQL queries.
4. SQL Server stores and retrieves data such as tickets, feedback, categories, subcategories, users, and roles.
5. The result is returned to the Razor View, and the user can see the updated information on the website.

Frontend–Backend Connectivity

The SupportSphere application follows the ASP.NET Core MVC architecture. Users interact with Razor Views through a web browser. The requests generated from the user interface are handled by MVC Controllers. Controllers process business logic and communicate with the database using Entity Framework Core. Entity Framework Core acts as an Object Relational Mapper (ORM) and performs database operations on SQL Server. After processing, the data is returned to the controller and displayed back to the user through Razor Views.

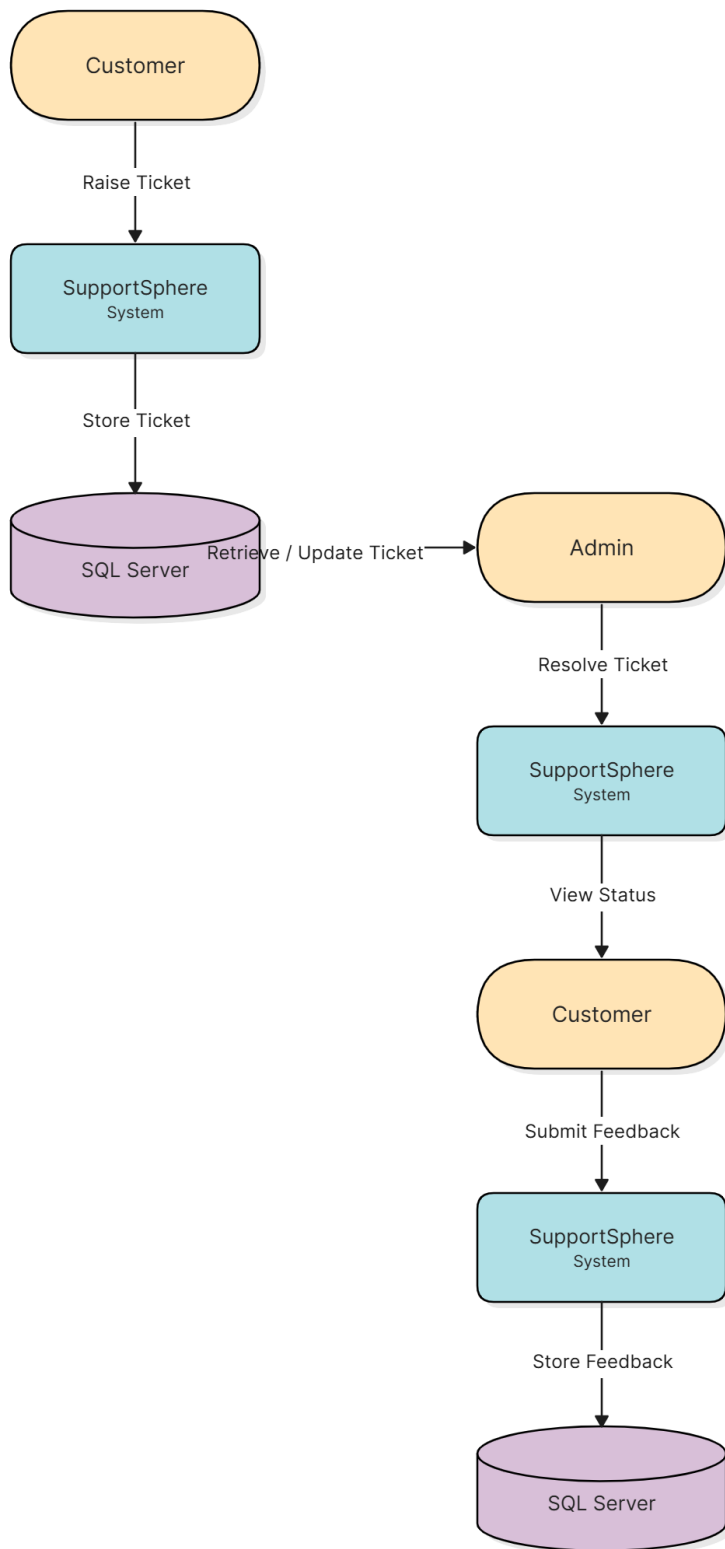
Workflow:

User → Razor View → Controller → Entity Framework Core → SQL Server → Controller → Razor View → User

Data Flow Diagram:

1. Customer logs into the system.
2. Customer raises a support ticket.
3. Ticket information is stored in SQL Server.
4. Administrator reviews and updates ticket status.
5. Customer views updated ticket information.
6. Customer submits feedback after issue resolution.
7. Feedback is stored and reviewed by administrators.

6. Data Flow Diagram (DFD)



7. Entity Relationship Diagram (ER Diagram)



Entities:

Category

- CategoryId (PK)
- CategoryName

SubCategory

- SubCategoryId (PK)
- SubCategoryName
- CategoryId (FK)

Ticket

- TicketId (PK)
- TicketNumber
- Title
- Description
- Status
- Priority
- CategoryId (FK)
- SubCategoryId (FK)
- CustomerId

Feedback

- FeedbackId (PK)
- Rating
- Comments
- TicketId (FK)

Relationships:

- Category → SubCategory (1:M)
 - Category → Ticket (1:M)
 - SubCategory → Ticket (1:M)
 - Ticket → Feedback (1:1)
-

8. Database Design

The project uses SQL Server as the relational database management system.

Database Tables

Categories

Stores ticket categories.

SubCategories

Stores ticket subcategories.

Tickets

Stores ticket information and status.

Feedbacks

Stores customer feedback and ratings.

AspNetUsers

Stores registered user information.

AspNetRoles

Stores role information.

AspNetUserRoles

Stores user-role mappings.

Database Features

- Primary Keys
 - Foreign Keys
 - Referential Integrity
 - Entity Framework Migrations
 - Relationship Mapping
-

9. Business Rules Summary

The following business rules are implemented within the system:

Rule 1

Every ticket must belong to a category.

Rule 2

Every ticket must belong to a subcategory.

Rule 3

Ticket title is mandatory.

Rule 4

Default ticket status is “Open”.

Rule 5

Default ticket priority is “Medium”.

Rule 6

Feedback ratings must be between 1 and 5.

Rule 7

Only administrators can access administrative functions.

Rule 8

Customers can raise, view, edit, and manage their own tickets.

Rule 9

Feedback can be associated with tickets.

Rule 10

Role-based access control is enforced using ASP.NET Identity.

10. Features Implemented

Customer Module

- Register
- Login
- Raise Tickets
- View Ticket History
- Edit Tickets
- Delete Tickets
- View Ticket Details
- Submit Feedback

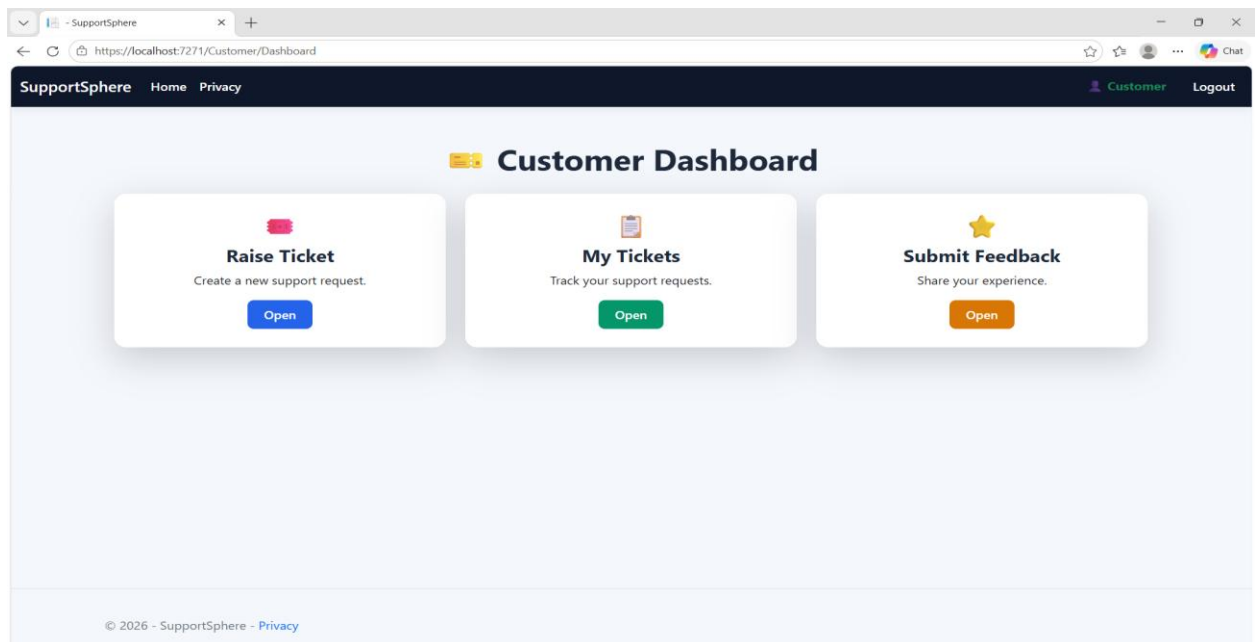


Fig 1:Customer Dashboard

SupportSphere Home Privacy Customer Logout

Raise New Ticket

CategoryId
-- Select Category --

SubCategoryId
-- Select Sub Category --

Title
-- Select Issue --

Description
Describe your issue in detail

Priority
Medium

[Submit Ticket](#) [Back to List](#)

© 2026 - SupportSphere - Privacy

Fig 2:Raise Ticket Page

SupportSphere Home Privacy Customer Logout

Index

TicketNumber	Title	Description	Status	Priority	CreatedDate	CustomerId	Category	SubCategory	
TKT6391613325766794	Password Reset Request		Open	High	04-06-2026 09:15:32	valli@gmail.com	Technical Issue	Password Reset	Edit View Delete
TKT639161683924525776	Password Reset Request		Open	Medium	04-06-2026 11:13:12	valli@gmail.com	Account Issue	Login Problem	Edit View Delete

© 2026 - SupportSphere - Privacy

Fig 3:My Tickets Page

Admin Module

Administrators can:

- Login
- View Dashboard
- Manage All Tickets
- Resolve Tickets
- Edit Ticket Information
- Delete Tickets
- View Customer Feedback
- View Reports

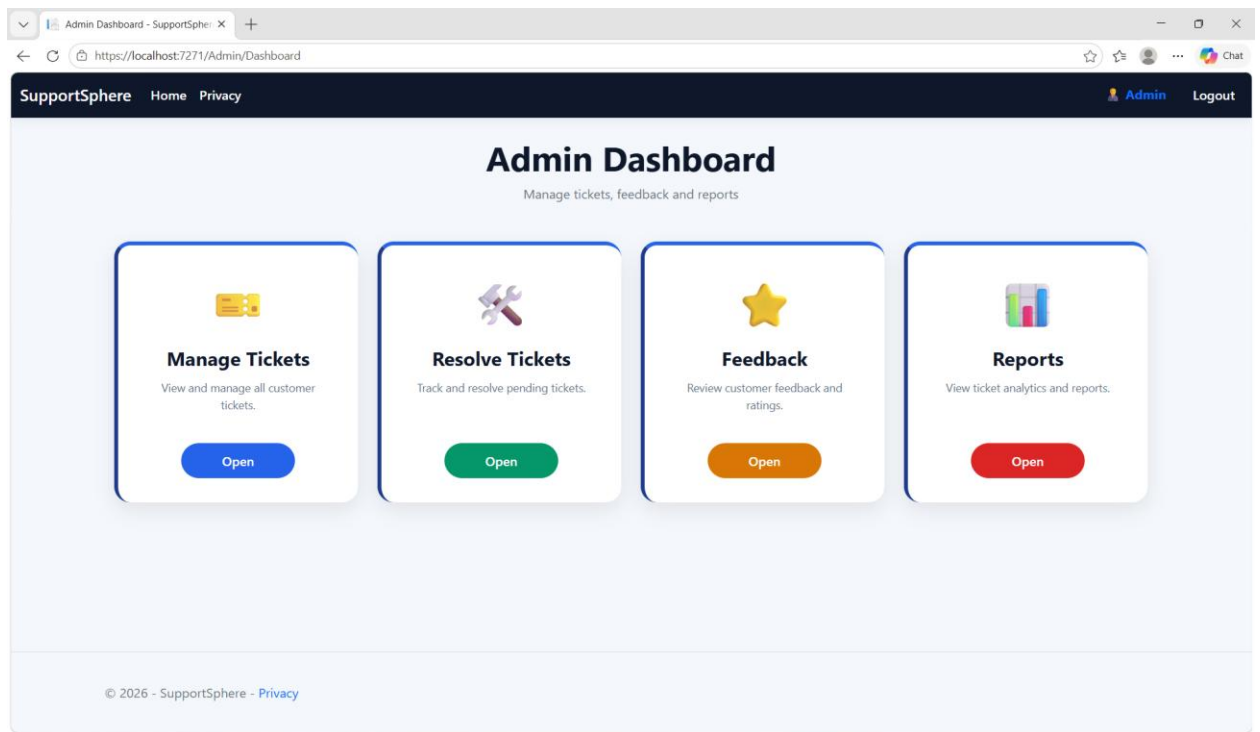


Fig 4:ADMIN DASHBOARD

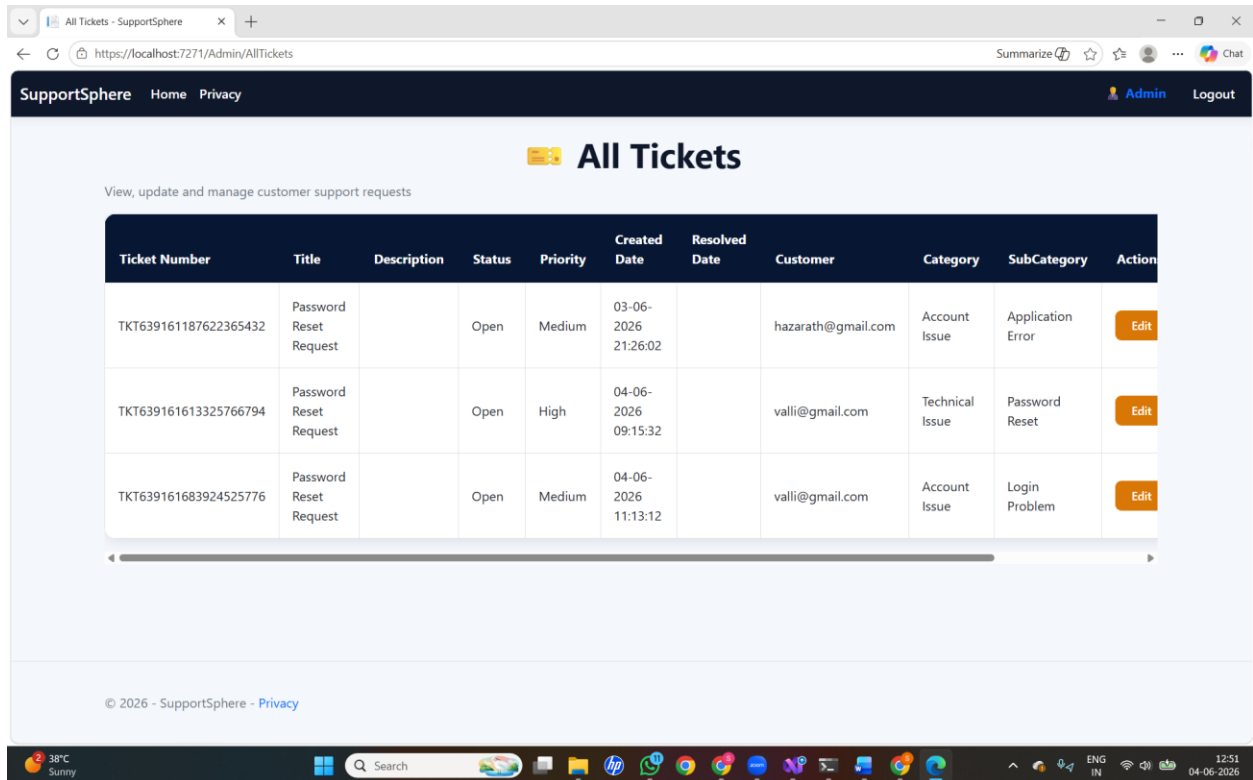


Fig 5:ALL TICKETS

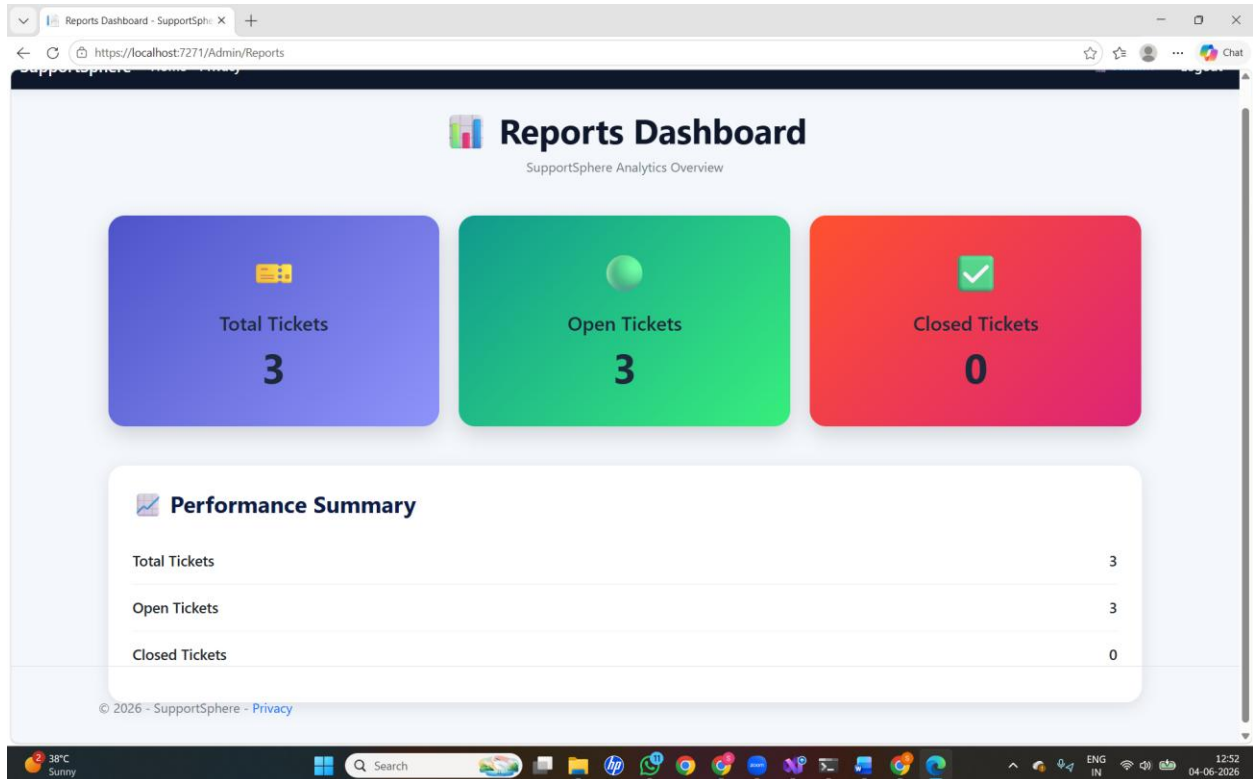


Fig 6:Report Dashboard

11. MVC Architecture Implementation

The project follows the ASP.NET Core MVC architecture.

Models

Represent business entities.

Examples:

- Ticket.cs
- Feedback.cs
- Category.cs
- SubCategory.cs

Views

Responsible for displaying information to users.

Examples:

- Login
- Register
- Dashboard
- Create Ticket
- Edit Ticket
- Details

Controllers

Handle requests and business logic.

Examples:

- TicketsController
- AdminController
- CustomerController
- FeedbacksController

12. Object-Oriented Programming Concepts Used

Encapsulation

Data is encapsulated within classes and accessed through properties.

Example:

- Ticket class
- Feedback class

Abstraction

Business entities are represented through models without exposing database implementation details.

Classes and Objects

Examples:

- Ticket
 - Feedback
 - Category
 - SubCategory
-

13. SOLID Principles And Design Patterns

Solid Principles

Single Responsibility Principle (SRP)

Each class is responsible for one purpose.

Examples:

- Ticket model stores ticket information.
- Feedback model stores feedback information.
- RoleSeeder handles role creation.
- Controllers handle request processing.

Dependency Inversion Principle (DIP)

Dependency Injection is used throughout the application.

Example:

ApplicationDbContext is injected into controllers rather than being instantiated directly.

Example:

Controller → ApplicationDbContext

Open Closed Principle (OCP) – Partial

The application was extended through API controllers without modifying existing MVC functionality.

Example:

TicketsApiController was added separately for API access.

Design Patterns

MVC Pattern

The project follows the Model-View-Controller architecture to separate user interface, business logic, and data access.

Dependency Injection Pattern

Controllers receive ApplicationDbContext through constructor injection, reducing tight coupling.

ORM Pattern

Entity Framework Core is used as an Object Relational Mapper to interact with SQL Server using C# objects.

Identity Pattern

ASP.NET Identity manages authentication, authorization, and user roles.

14. Validation and Security

Model Validation

Data Annotations are used:

- Required
- StringLength
- Range

Examples:

- Ticket Title validation
- Feedback Rating validation

Authentication

Implemented using ASP.NET Core Identity.

Features:

- Registration
- Login
- Password Management

Authorization

Role-Based Access Control:

- Admin
- Customer

Security Features

- ASP.NET Identity
- Authorization Attributes
- Validation Controls
- HTTPS Support

User Validation through Authentication and Authorization

SupportSphere uses ASP.NET Core Identity for secure user management.

Authentication:

- Users register using email and password.
- Login credentials are validated by ASP.NET Identity.
- Authenticated users gain access to protected pages.

Authorization:

- Role-based authorization is implemented.
- Two roles are available:
 - Admin
 - Customer
- Admin users can manage tickets, reports, and feedback.
- Customer users can create and track their own support tickets.

Benefits:

- Prevents unauthorized access.
 - Ensures data security.
 - Restricts administrative operations to authorized users only.
-

15. REST API and OpenAPI Documentation

API Endpoint

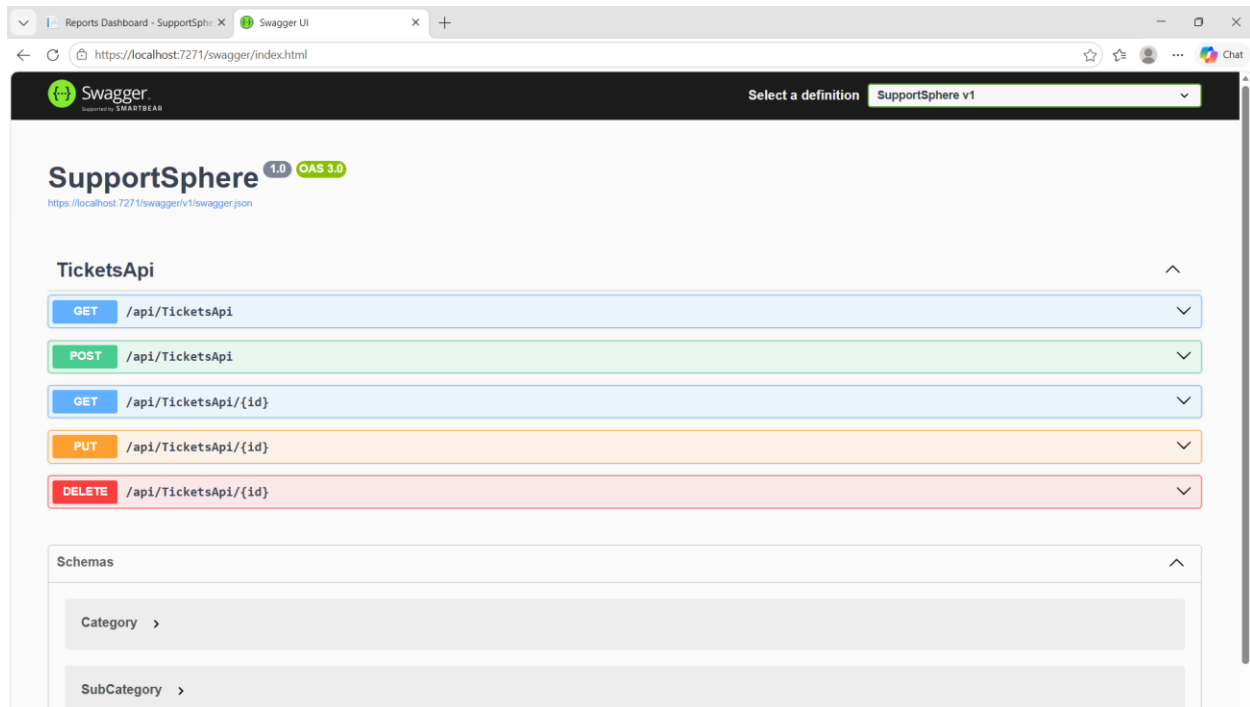
Returns all tickets in JSON format.

Additional Operations:

- GET
- GET BY ID
- POST
- PUT
- DELETE

Swagger/OpenAPI

Swagger is integrated to automatically generate API documentation and allow API testing through the browser.



16. Unit Testing

Test Classes

TicketTests

- Default Status Validation
- Default Priority Validation
- Ticket Title Validation
- Ticket Number Validation
- Created Date Validation

FeedbackTests

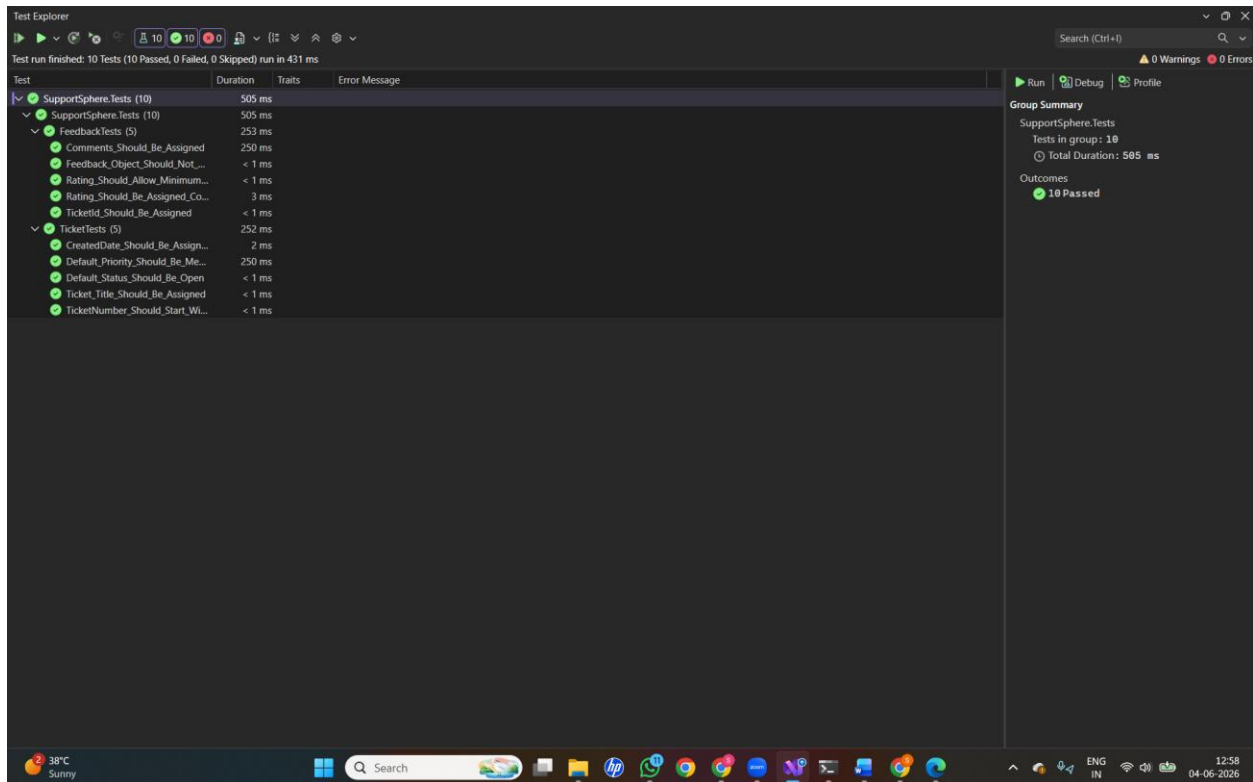
- Rating Validation
- Comments Validation
- Ticket Association Validation

Test Results

Total Tests: 10

Passed: 10

Failed: 0



17. Exception Handling

ASP.NET Core exception handling middleware is used.

Features:

- Error Pages
- Validation Messages
- Runtime Exception Handling

Benefits:

- Improved reliability
- Better user experience
- Easier debugging

18. Scope for Future Development

Given additional development time, the following enhancements can be implemented:

- JWT Authentication
- Email Notification System
- SMS Notifications
- Cloud Deployment on Azure
- Docker Containerization
- CI/CD Pipeline using GitHub Actions
- Advanced Reporting Dashboard
- Real-Time Ticket Tracking
- AI-based Ticket Classification
- Mobile Application Integration

These enhancements would improve scalability, usability, and enterprise readiness.

19. Conclusion

SupportSphere successfully automates customer support operations through a secure and scalable ticket management platform. The application demonstrates the practical implementation of ASP.NET Core MVC, Entity Framework Core, SQL Server, ASP.NET Identity, REST API development, Swagger/OpenAPI documentation, and xUnit testing.

The system provides efficient ticket tracking, role-based access control, customer feedback management, and administrative monitoring while maintaining security, maintainability, and scalability.